ДИСЦИПЛИНА «ИНФОРМАТИКА», 1 СЕМЕСТР

Лабораторная работа №2

«Массивы и строки»

Задания и справочные материалы для выполнения на C# (Microsoft Visual Studio)

ЧАСТЬ I · ЗАДАНИЯ

Цель: целью данной работы является получение опыта работы с массивами и строками на языке высокого уровня C#, в среде программирования Microsoft Visual Studio.

Задание 1:

На вход программы подаётся массив целых чисел произвольной длины (из консоли или заполняется циклом). Далее происходит инвертирование массива, после чего в консоль выводятся исходный и инвертированный массивы.

Пример:

- **Вход:**
  | 1 | 2 | 3 | 4 | 5 |
- **Выход:**
  | 5 | 4 | 3 | 2 | 1 |

Массив — это пронумерованная коллекция элементов одного типа. Массивы используются для хранения и обработки большого количества данных одного типа. Они имеют фиксированный размер и индексируются с нуля.

Инвертирование массива означает изменение порядка элементов массива так, чтобы первый элемент становился последним, второй элемент — вторым с конца и так далее.

Для выполнения 1 задания вам понадобятся операторы:
1. `Console.ReadLine` – для считывания числа из консоли
2. `Console.WriteLine` – для вывода на экран сообщений
3. `int` – Объявление массива целых чисел
4. `for` – цикл, в котором некоторая результирующая переменная будет изменяться (например, увеличиваться или уменьшаться) определенное количество раз.
5. Алгоритм «классической» замены порядка элементов местами (обмен значений первого и последнего, второго и предпоследнего элементов и так далее через временную переменную) — подробное описание алгоритма и пример кода приведены в справочном материале.

Задание 2:

На вход программы подаётся размерность двумерного массива MxN, где M обозначает количество строк, а N — количество столбцов в массиве (значения M и

N заданы в коде). Далее двумерный массив заполняется числами от 1 до M*N “змейкой”, после чего заполненный массив выводится в консоль.

Пример:
Пример:
* **Вход:** M = 3, N = 3;
* **Выход:**
1	2	3
6	5	4
7	8	9

Двумерный массив — это массив, состоящий из одномерных массивов (строк), которые расположены последовательно в памяти. Он представляет собой матрицу, состоящую из строк и столбцов.

Для выполнения 2 задания вам понадобятся операторы:
1. `Console.ReadLine` – для считывания числа из консоли
2. `Console.WriteLine` – для вывода на экран сообщений
3. `int` – Объявление массива целых чисел
4. `for` – цикл, в котором некоторая результирующая переменная будет изменяться определенное количество раз.
5. `if, else` – используются для управления направлением движения "змейки" по массиву.

Задание 3:

На вход программы подаётся размерность двумерного массива MxN и одномерный массив размерностью M*N (значения M и N заданы в коде, массив заполняется в коде или из консоли). Далее создаётся двумерный массив размерностью MxN и элементы одномерного массива построчно копируются в двумерный массив, после чего оба массива выводятся в консоль.

Пример:
* *Вход: M = 3, N = 3;* `[1, 2, 3, 4, 5, 6, 7, 8, 9]`
* *Выход:**
1	2	3
4	5	6
7	8	9

Подсказки:
Для выполнения 3 задания вам понадобятся операторы:

1. `Console.ReadLine` – для считывания числа из консоли
2. `Console.WriteLine` – для вывода на экран сообщений
3. `int` – Объявление массива целых чисел
4. `for` – цикл, в котором некоторая результирующая переменная будет изменяться определенное количество раз.

Задание 4:

На вход программы подаётся двумерный массив размерностью MxN (значения M и N заданы в коде). Далее осуществляется поиск максимального и минимального элементов в массиве, после чего массив, а также значения максимального и минимального элементов выводятся в консоль.

Пример:

- **Вход:** M = 3, N = 3;
1	2	3
4	5	6
7	8	9
- **Выход:** MIN = 1, MAX = 9;

Для выполнения 4 задания вам понадобятся операторы:
1. `Console.ReadLine` – для считывания числа из консоли
2. `Console.WriteLine` – для вывода на экран сообщений
3. `int` – Объявление массива целых чисел
4. `for` – цикл, в котором некоторая результирующая переменная будет изменяться определенное количество раз.
5. `Random` – в работе этого класса заложен алгоритм, который генерирует последовательность чисел.

Задание 5:

На вход программы подаётся строка символов (задана в коде или из консоли). Далее определяется количество разных символов в строке, после чего в консоль выводится заданная строка и количество разных символов в ней.

Пример:

- **Вход:** “Это тестовая строка.”
- **Выход:** Symbols = 12;

Для выполнения 5 задания вам понадобятся операторы:
1. `Console.ReadLine` – для считывания числа из консоли
2. `Console.WriteLine` – для вывода на экран сообщений
3. `foreach` - используется для итерации по символам в строке

6. На вход программы подаётся строка символов (заполняется в коде или из консоли). Далее определяется количество повторений для каждого символа, после чего в консоль выводится заданная и уникальные символы этой строки с количеством их повторений в ней.

Пример:

- **Вход:** “мама мыла раму”
- **Выход:** М = 4, А = 4, Ы = 1, Л = 1, Р = 1, У = 1;

Для выполнения 6 задания вам понадобятся операторы:
1. `Console.ReadLine` – для считывания числа из консоли
2. `Console.WriteLine` – для вывода на экран сообщений
3. `int` – Объявление массива целых чисел (или `Dictionary` для удобства)
4. `foreach` – используется для итерации по символам в строке
5. `TryGetValue` – получает из словаря элемент по ключу key.

Задание 7:

На вход программы подаётся строка символов (заполняется в коде или из консоли). Далее пользователь вводит символ, и введённый пользователем символ удаляется из строки, после чего получившаяся строка выводится в консоль.

Пример:

- **Вход:** “исходная строка”, “а”
- **Выход:** “исходня строк”

Для выполнения 7 задания вам понадобятся операторы:
1. `Console.ReadLine` – для считывания числа из консоли
2. `Console.WriteLine` – для вывода на экран сообщений
3. Самостоятельная реализация удаления символа из строки (без использования готовых функций): построение новой строки, в которую копируются все символы исходной строки, кроме удаляемого — подробное описание алгоритма и пример кода приведены в справочном материале.

Задание 8:

На вход программы подается двумерныый массив целых чисел `int[N, N], N = 2500-10_000`. Реализовать два прохода по всем элементам: по строкам (внешний цикл по

`i`, внутренний по `j`) и по столбцам (внешний по `j`, внутренний по `i`), замерить время каждого прохода через `Stopwatch`.

Вопросы

1. Какой вариант цикла быстрее проходит по всему циклу?
2. Меняется ли общее количество итераций цикла?

ЧАСТЬ II · СПРАВОЧНАЯ ИНФОРМАЦИЯ

Общие сведения, необходимые для выполнения заданий

Все задания выполняются в виде консольного приложения C# (Console App) в Visual Studio. Любая программа начинается с точки входа — метода `Main`:

```
using System;

class Program
{
    static void Main()
    {
        // здесь пишется код программы
    }
}
```

Метод `Console.ReadLine()` всегда возвращает **строку** (тип `string`), даже если пользователь ввёл число. Поэтому, если введённое значение нужно использовать как число, его необходимо преобразовать при помощи `int.Parse` или `Convert.ToInt32`:

```
Console.WriteLine("Введите число:");
string input = Console.ReadLine();  // input содержит строку, например "5"
int number = int.Parse(input);      // number содержит число 5

// то же самое можно записать в одну строку:
int number2 = int.Parse(Console.ReadLine());
// или
int number3 = Convert.ToInt32(Console.ReadLine());
```

Это понадобится в большинстве заданий: как только речь идёт о вводе размера массива, отдельного числа или индекса из консоли — их сначала считывают как строку, а затем преобразуют в `int`.

Объявление массивов

Объявление массивов может выглядеть следующим образом:

```
int[] mas = new int[10];      // одномерный массив на 10 элементов
int[,] mas2D = new int[3,3]; // двумерный массив размером 3x3
```

Инициализация массивов

Можно инициализировать массивы при объявлении:

```
int[] m1 = new int[5] { 1, 2, 3, 4, 5 }; // массив из 5 целых чисел
float[] m2 = { 1.0f, 2.0f, 3.0f };      // массив из 3 вещественных чисел
int[,] m2d = { {1, 2}, {3, 4}, {5, 6} }; // объявление и инициализация двумерного
массива
```

Либо используя циклы:

```
int[] m = new int[10]; // массив целых чисел из 10 элементов
for (int i = 0; i < 10; i++) // заполнение числами от 1 до 10 (индексы от 0 до 9)
{
    m[i] = i + 1;
}

int[,] m2d = new int[3,3]; // массив целых чисел из 9 элементов
// заполнение строк массива числами (например, 1 2 3 / 1 2 3 / 1 2 3)
for (int i = 0; i < 3; i++)
{
    for (int j = 0; j < 3; j++)
    {
        m2d[i,j] = j + 1;
    }
}
```

Считывание массива чисел из консоли

Если массив нужно не заполнять в коде, а получать от пользователя, элементы считываются по очереди в цикле, каждый раз с преобразованием строки в число:

```
Console.WriteLine("Введите количество элементов массива:");
int n = int.Parse(Console.ReadLine());

int[] m = new int[n];
for (int i = 0; i < n; i++)
{
    Console.WriteLine("Введите элемент {0}:", i + 1);
    m[i] = int.Parse(Console.ReadLine());
}
```

Вывод значений массива

Вывод значений массива на экран можно реализовать при помощи цикла `for`:

```
int[] m = { 1, 2, 3, 4, 5 };
for (int i = 0; i < m.Length; i++)
{
    Console.Write("{0} ", m[i]); // вывод на экран элемента массива и пробела
}
```

Вывод матрицы в консоль

```
// Выводим матрицу на консоль
Console.WriteLine("Матрица:");
for (int i = 0; i < rows; i++)
{
    for (int j = 0; j < cols; j++)
    {
        Console.Write(matrix[i, j] + " ");
    }
    Console.WriteLine(); // Переход на новую строку
}
```

Алгоритм инвертирования (разворота) массива

Инвертирование ("разворот") массива — это изменение порядка его элементов на обратный: первый элемент становится последним, второй — предпоследним и так далее. Классический способ сделать это без вспомогательного массива — **обмен элементов местами**, начиная с двух концов массива и двигаясь к середине.

Идея алгоритма:
1. Заводится временная переменная `temp`, в которой будет на мгновение сохраняться значение одного из элементов на время обмена (без неё, при прямом присваивании `m[i] = m[j]`, исходное значение `m[i]` было бы потеряно).
2. Два "указателя"-индекса — один начинается с начала массива (`i = 0`), другой — с конца (`m.Length - 1 - i`) — двигаются друг к другу.
3. На каждом шаге элементы, на которые указывают эти индексы, меняются местами.
4. Цикл выполняется `m.Length / 2` раз — ровно столько пар нужно поменять местами; если элементов нечётное количество, средний элемент не с кем менять, и он автоматически остаётся на своём месте.

```
int[] m = { 1, 2, 3, 4, 5 };
int temp;

for (int i = 0; i < m.Length / 2; i++)
{
    temp = m[i];                    // временно сохраняем значение элемента с начала массива
    m[i] = m[m.Length - 1 - i];     // на его место записываем значение "зеркального" элемента с конца
    m[m.Length - 1 - i] = temp;     // на место "зеркального" элемента записываем сохранённое значение
}
// теперь m содержит { 5, 4, 3, 2, 1 }
```

Обратите внимание: в отличие от примера ниже (разворот строки), здесь **не создаётся новый массив** — элементы меняются местами внутри исходного массива. Если по условию задания нужно вывести и исходный, и инвертированный массив, то перед разворотом исходные значения нужно вывести на экран (или заранее сохранить их копию), поскольку после выполнения цикла исходный массив будет уже изменён.

Работа со строками

Строка может быть объявлена и инициализирована следующим образом:

```
string str;             // создание строковой переменной
str = "Hello World";    // запись текста в строковую переменную
```

Считать строку из консоли можно при помощи метода `ReadLine`:

```
string str;
str = Console.ReadLine(); // создание строковой переменной и запись в нее текста из консоли
```

В сущности, строка является массивом символов, поэтому к ней применимы операции поэлементного доступа:

```
Console.WriteLine(str);     // вывод на экран всей строки
Console.WriteLine(str[3]);  // вывод на экран 4-го символа строки
```

Так же, к строке можно добавлять символы. Например, функцию создания инвертированной версии строки можно реализовать так:

```
string str;
str = Console.ReadLine(); // запись текста в строковую переменную
string strInv = "";       // создание пустой строки

// цикл, начинающийся в конце строки и идущий к её началу
for(int i = str.Length-1; i >= 0; i--)
{
    strInv += str[i]; // добавление символа к пустой строке
}
```

Здесь используется другой подход, чем в алгоритме разворота массива выше: строки в C# неизменяемы (нельзя поменять символ внутри существующей строки местами, как элемент массива), поэтому вместо обмена элементов местами строится **новая** строка, в которую символы добавляются в обратном порядке — от последнего к первому.

Удаление символа из строки без использования готовых функций

Чтобы удалить из строки все вхождения определённого символа самостоятельно (не используя готовую функцию замены), можно применить тот же приём, что и при развороте строки: построить новую строку, но при этом пропускать (не копировать в неё) тот символ, который нужно удалить.

Алгоритм:
1. Создаётся пустая строка-результат.
2. Цикл `foreach` перебирает все символы исходной строки по очереди.
3. Каждый символ сравнивается с тем, который нужно удалить.
4. Если символ **не совпадает** с удаляемым — он добавляется в строку-результат; если совпадает — просто пропускается (не добавляется).

```
string str = "исходная строка";
char target = 'a';     // символ, который нужно удалить
string result = "";    // результирующая строка

foreach (char c in str)
{
    if (c != target)    // если текущий символ не совпадает с удаляемым
    {
        result += c;    // копируем его в результирующую строку
    }
    // если c == target, ничего не делаем — символ пропускается
}
// result содержит "исходня строк"
```

Словари (Dictionary)

Словарь (`Dictionary`) — это коллекция пар «ключ — значение» (`key – value`), где каждому ключу соответствует ровно одно значение, а сами ключи не повторяются. Словарь удобен, когда нужно быстро находить значение по некоторому «идентификатору» — например, узнавать количество повторений конкретного символа по самому символу.

Объявление словаря, где ключ — символ (`char`), а значение — число повторений (`int`):

```
Dictionary<char, int> counts = new Dictionary<char, int>();
```

Добавить пару «ключ — значение» и проверить, есть ли уже такой ключ, можно через `ContainsKey` :

```
foreach (char c in str)
{
    if (counts.ContainsKey(c))   // если такой символ уже встречался и есть в словаре
    {
        counts[c]++;              // увеличиваем счётчик повторений на 1
    }
    else
    {
        counts.Add(c, 1);         // добавляем новый ключ со значением 1 (первая
встреча символа)
    }
}
```

То же самое можно сделать компактнее при помощи `TryGetValue` , который одновременно проверяет наличие ключа и возвращает его значение (если ключа нет — возвращает `false` , а `value` устанавливается в 0):

```
Dictionary<char, int> counts = new Dictionary<char, int>();

foreach (char c in str)
{
    if (counts.TryGetValue(c, out int value))  // пытаемся получить текущее значение по ключу c
    {
        counts[c] = value + 1;                 // если ключ был найден — увеличиваем его значение
    }
    else
    {
        counts[c] = 1;                         // если ключа не было — заводим новый со значением 1
    }
}
```

Перебрать все пары «ключ — значение» словаря и вывести их можно через `foreach` по `KeyValuePair`:

```
foreach (KeyValuePair<char, int> pair in counts)
{
    Console.WriteLine("{0} = {1}", pair.Key, pair.Value);
}
```

Класс Random

Класс `Random` используется, когда значения массива нужно не вводить вручную, а сгенерировать случайно (например, чтобы быстро получить массив для поиска максимума и минимума):

```
Random rnd = new Random();          // создаём генератор случайных чисел
int value = rnd.Next(1, 100);       // случайное целое число от 1 (включительно) до 100 (не включая)

int[] m = new int[10];
for (int i = 0; i < m.Length; i++)
{
    m[i] = rnd.Next(1, 100);        // заполняем массив случайными числами
}
```

Класс Stopwatch

Класс `Stopwatch` (из пространства имён `System.Diagnostics`) позволяет замерить, сколько времени выполняется участок кода — например, чтобы сравнить скорость

проходов по массиву по строкам и по столбцам:

```
using System.Diagnostics;

Stopwatch sw = new Stopwatch();

sw.Start();
// сюда помещается код, время выполнения которого нужно измерить
sw.Stop();

Console.WriteLine("Время выполнения: {0} мс", sw.ElapsedMilliseconds);
```

Чтобы замер был честным, `Stopwatch` нужно запускать и останавливать отдельно для каждого варианта прохода (по строкам и по столбцам), не забывая, что оба прохода должны обрабатывать один и тот же массив.

Ссылки на описания используемых функций/операторов:

1. **Console.ReadLine, Console.WriteLine, Console.ReadKey** - `https://devpractice.ru/c-sharp-lesson-3-console-io/#p33`
2. **if** - `https://devpractice.ru/c-sharp-lesson-6-if-and-for/`
3. **for** - `https://metanit.com/sharp/tutorial/2.6.php`
4. **int, int.Parse, Convert.ToInt32** - `https://docs.microsoft.com/en-us/dotnet/api/system.int32.parse`
5. **else** - `https://metanit.com/sharp/tutorial/2.5.php`
6. **foreach** - `https://metanit.com/sharp/tutorial/4.9.php`
7. **Dictionary** – `https://docs.microsoft.com/en-us/dotnet/api/system.collections.generic.dictionary-2`
8. **TryGetValue** – `https://docs.microsoft.com/en-us/dotnet/api/system.collections.generic.dictionary-2.trygetvalue`
9. **Random** – `https://docs.microsoft.com/en-us/dotnet/api/system.random`
10. **Stopwatch** – `https://docs.microsoft.com/en-us/dotnet/api/system.diagnostics.stopwatch`